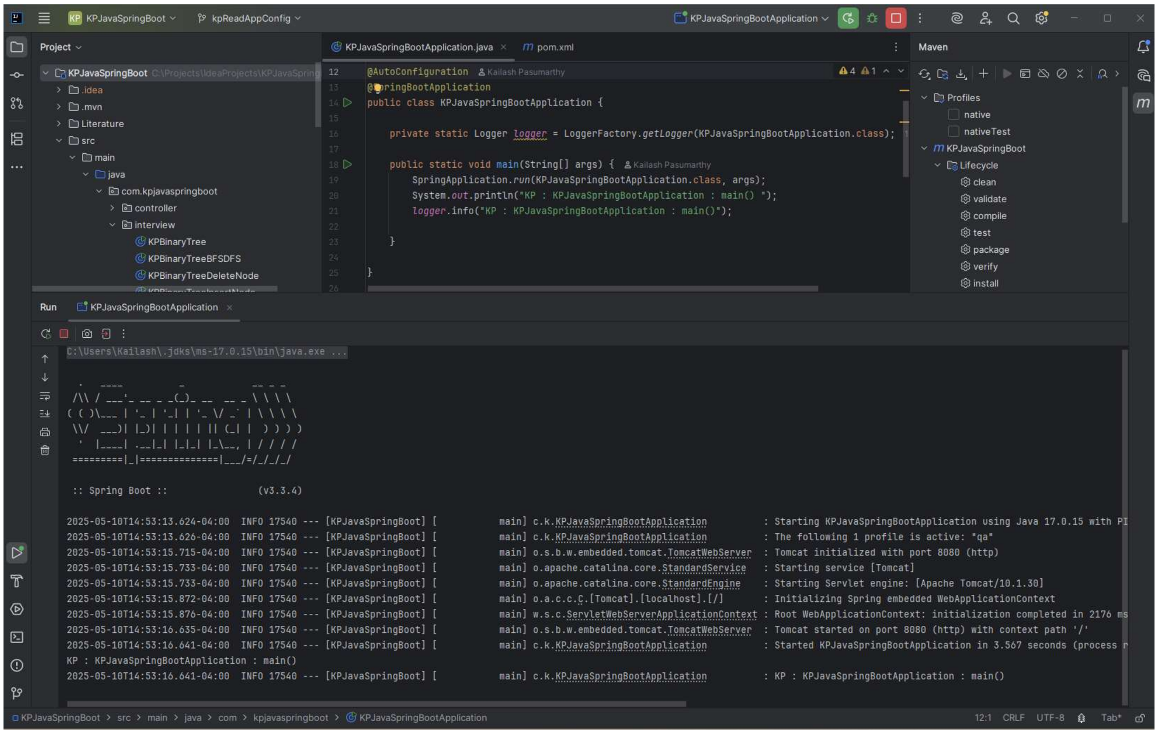
KPJavaSpringBoot-Docker

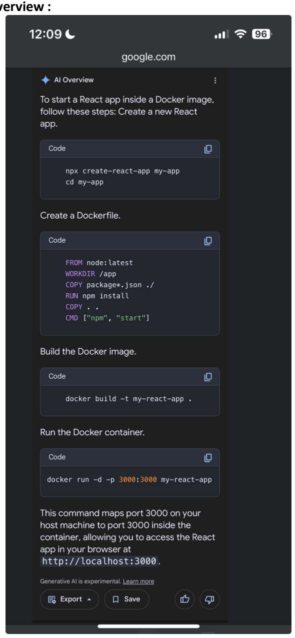
Saturday, May 10, 2025 2:51 PM

KPJavaSpringBootApp

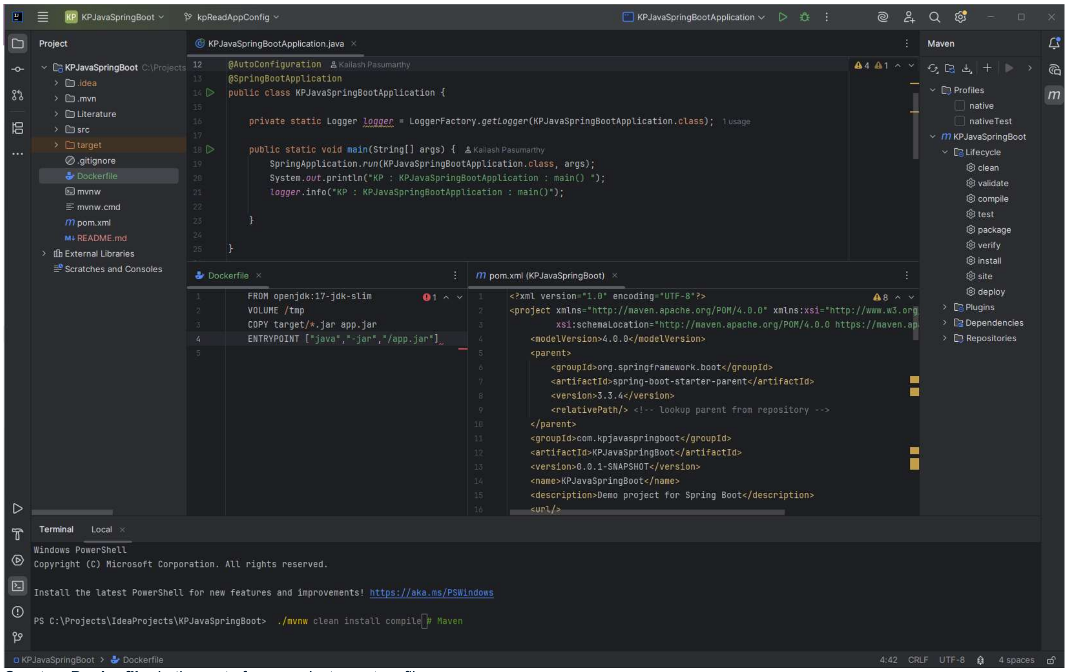


KPJavaSpringBootApp : Docker Containerize

- Steps to run KPJavaSpringBootApp inside a Docker Container ..
- **AI Overview :**



- To run a Java Spring Boot application in a Docker container, follow these steps:
  - **Create a Spring Boot application:**
    .Opens in new tab
    If you don't have one, you can create a sample application using Spring Initializer.
  - Use KPJavaSpringBootApp : https://github.com/KPasumarthy/KPJavaSpringBoot.git
  - **Build the application:**
    .Opens in new tab
    Use Maven or Gradle to build your Spring Boot application. This will generate a .jar file in the target directory.
    - Code
      ./mvnw clean install *# Maven*
      ./gradlew build # Gradle

- 

- **Create a Dockerfile:** In the root of your project, create a file named Dockerfile with the following content:
  Code

  ```
  FROM openjdk:17-jdk-slim
  VOLUME /tmp
  COPY target/*.jar app.jar
  ENTRYPOINT ["java","-jar","/app.jar"]
  ```
- **Build the Docker image:** Navigate to the directory containing your Dockerfile and run the following command to build the Docker image:
  Code

  ```
  docker build -t your-image-name .
  ```

  Replace your-image-name with a suitable name for your image.

- **Run the Docker container:** Once the image is built, you can run a container based on this image using the following command:
  Code

  ```
  docker run -p 8080:8080 your-image-name
  ```

  The -p argument establishes a port mapping from the host's port 8080 to the container's port 8080.
- **Verify the application:** Open a web browser or use curl to access your application at http://localhost:8080. You should see the output from your Spring Boot application.

From <https://www.google.com/search?q=how+to+run+java+spring+boot+application+in+docker+container&sca_esv=c08a515a6aaa8d23&sxsrf=AHTn8zq_vs6iExUemMrzAmHo9DbP3AkWug%3A1746903556337&source=hp&ei=BKIfaMX3EZmnptQP8oqNsA0&iflsig=ACkRmUkAAAAAaB-wFFwwr1KqRQaRmP2AnMBVO_mJHv_G&oq=how+to+run+java+spring+boot+application+in+Docker+&gs_lp=Egdnd3Mtd2l6IiJob3cgdG8gcnVuIGphdmEgc3ByaW5nIGJvb3QgYXBwbGljYXRpb24gaW4gRG9ja2VyICoCCAAyBRAhGKABMgUQIRigATIFECEYoAEyBRAhGKABMgUQIRigATIFECEYnwUyBRAhGJ8FMgUQIRifBTIFECEYnwUyBRAhGJ8FSLWfAVAAWIKEAXAEeACQAQCYAX6gAagYqgEEMzUuM7gBA8gBAPgBAZgCKqACpRrCAgQQIxgnwgILEAAYgAQYkQIYigXCAgsQLhiABBixAxiDAcICBRAuGIAEwgIOEC4YgAQYsQMY0QMYxwHCAgsQLhiABBjRAxjHAcICChAjGIAEGCcYigXCAhMQLhiABBjHARgnGIoFGI4FGK8BwgIIEAAYgAQYsQPCAg4QABiABBiRAhixAxiKBcICCxAAGIAEGLEDGIMBwgIOEC4YgAQYxwEYjgUYrwHCAgQQABgDwgIIEC4YgAQYsQPCAg4QABiABBixAxiDARiKBcICBRAAGIAEwgILEC4YgAQYxwEYrwHCAgoQABiABBgUGIcCwgIHEAAYgAQYDcICBhAAGBYYHsICCxAAGIAEGIYDGIoFwgIIEAAYFhgKGB7CAggQABiABBiiBMICCBAAGKIEGIkFwgIFECEYqwKYAwCSBwQzNS43oAeE1gKyBwQzMS43uAeMGg&sclient=gws-wiz>

-

- Steps to start ReactJS App as a Docker Image inside Docker Container :
  - To start a React app inside a Docker image, follow these steps: Create a new React app.

    Code :
    ```
    npx create-react-app my-app
    cd my-app
    ```
  - Create a Dockerfile: In the root directory of your React app, create a file named Dockerfile (without any extension) and add the following content:
    Code
    ```
    FROM node:alpine
    WORKDIR /app
    COPY package*.json ./
    RUN npm install
    COPY . .
    CMD ["npm", "start"]
    ```
  - Build the Docker image.

    Code
    ```
    docker build -t my-react-app .
    ```
  - Run the Docker container.
  - Code

- docker run -d -p 3000:3000 my-react-app
- This command maps port 3000 on your host machine to port 3000 in the container, allowing you to access the React app in your browser at http://localhost:3000.

KPReactJSDocker : Docker Containerize KPReactJS

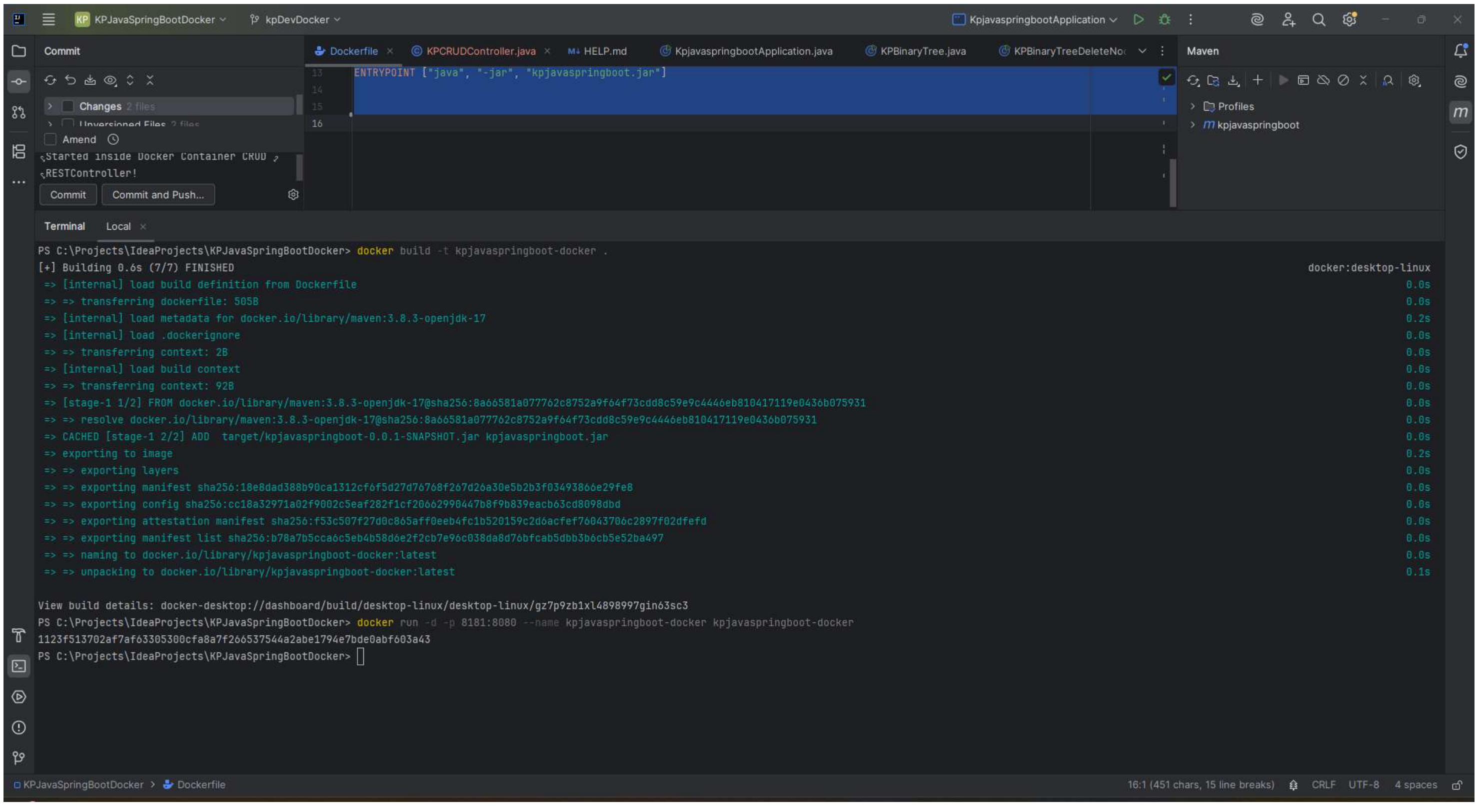
- How do I run a container?
- Docker ReadMe First Java Spring Boot App running inside a Docker Container..

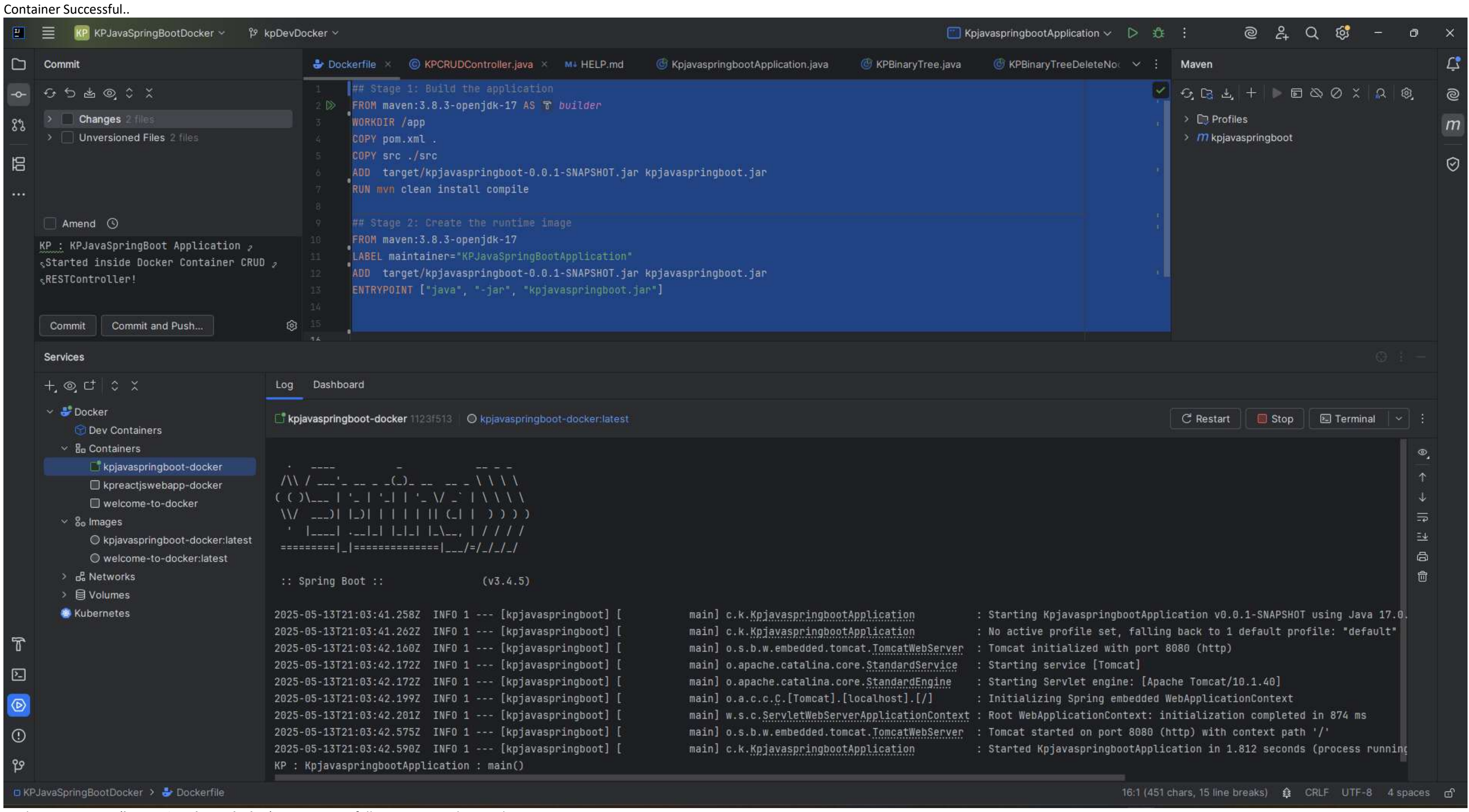
```
# Getting Started

### Reference Documentation
For further reference, please consider the following sections:

* [Official Apache Maven documentation](https://maven.apache.org/guides/index.html)
* [Spring Boot Maven Plugin Reference Guide](https://docs.spring.io/spring-boot/3.4.5/maven-plugin)
* [Create an OCI image](https://docs.spring.io/spring-boot/3.4.5/maven-plugin/build-image.html)
* [Spring Web](https://docs.spring.io/spring-boot/3.4.5/reference/web/servlet.html)
* [Spring Configuration Processor](https://docs.spring.io/spring-boot/3.4.5/specification/configuration-metadata/annotation-processor.html)

### Guides
The following guides illustrate how to use some features concretely:

* [Building a RESTful Web Service](https://spring.io/guides/gs/rest-service/)
* [Serving Web Content with Spring MVC](https://spring.io/guides/gs/serving-web-content/)
* [Building REST services with Spring](https://spring.io/guides/tutorials/rest/)

### Maven Parent overrides

Due to Maven's design, elements are inherited from the parent POM to the project POM.
While most of the inheritance is fine, it also inherits unwanted elements like `<license>` and `<developers>` from the parent.
To prevent this, the project POM contains empty overrides for these elements.
If you manually switch to a different parent and actually want the inheritance, you need to remove those overrides.

# KPJavaSpringBoot

# Welcome to KPJavaSpringBoot & Docker
https://www.baeldung.com/spring-boot-fix-the-no-main-manifest-attribute


# Welcome to KPJavaSpringBoot & Docker

This is a repo for new users getting started with Spring Boot Application running inside a Docker Container.

# Building

Maintainers should see [MAINTAINERS.md](MAINTAINERS.md).

Build and run:
```
docker build -t kpjavaspringboot-docker .
docker run -d -p 8181:8080 --name kpjavaspringboot-docker kpjavaspringboot-docker
```
Open `http://localhost:8181` in your browser.
```

- **Create Dockerfile inside the KPReactJSWebApp**
- ## Stage 1: Build the application

```
## Stage 1: Build the application
FROM maven:3.8.3-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
ADD  target/kpjavaspringboot-0.0.1-SNAPSHOT.jar kpjavaspringboot.jar
RUN mvn clean install compile

## Stage 2: Create the runtime image
FROM maven:3.8.3-openjdk-17
LABEL maintainer="KPJavaSpringBootApplication"
ADD  target/kpjavaspringboot-0.0.1-SNAPSHOT.jar kpjavaspringboot.jar
ENTRYPOINT ["java", "-jar", "kpjavaspringboot.jar"]
```

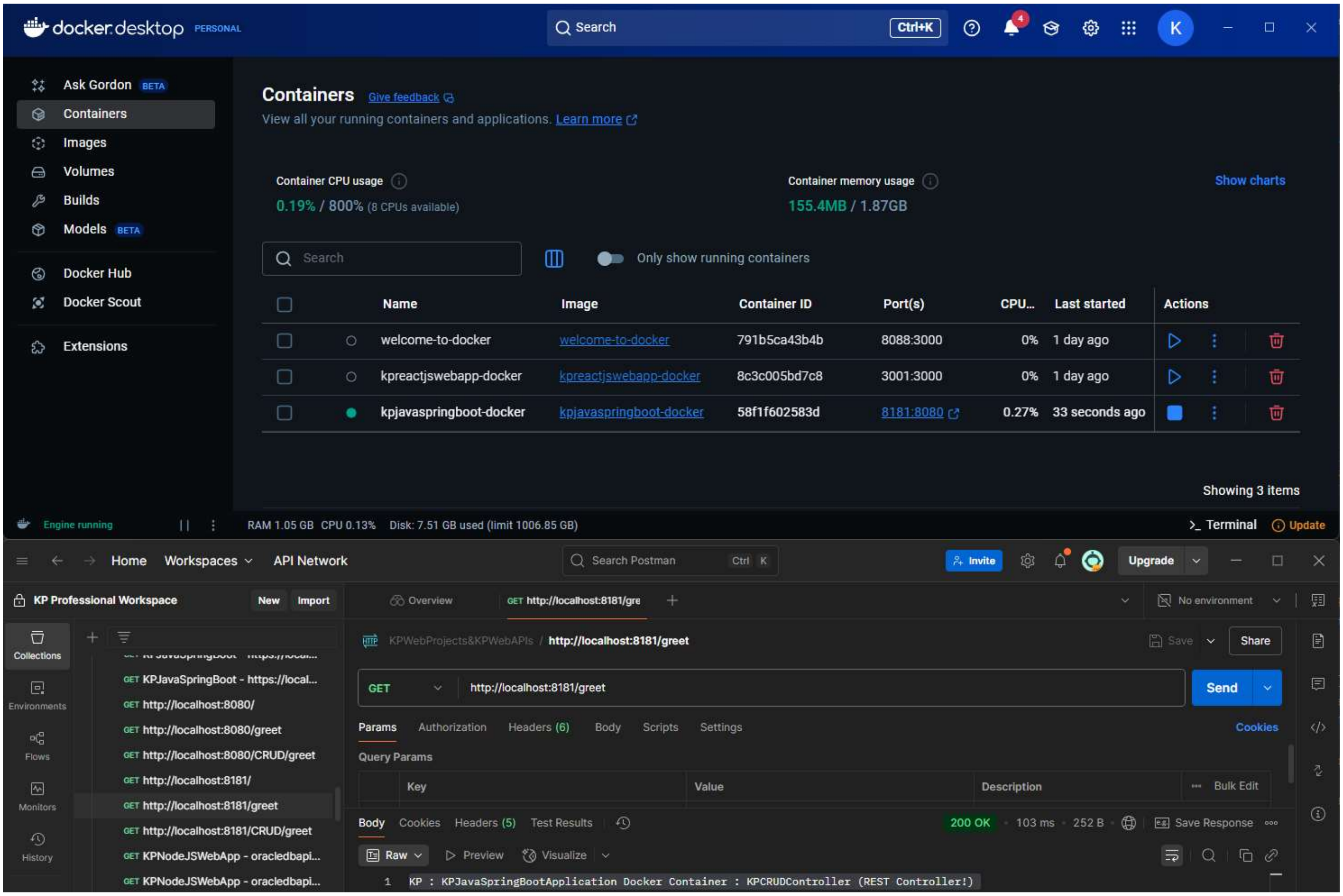
- Docker Build "kpjavaspringboot" app.. With command **"docker build -t** kpjavaspringboot-**docker ."**
- docker build -t kpjavaspringboot-docker .
- docker run -d -p 8181:8080 --name kpjavaspringboot-docker kpjavaspringboot-docker

- 

- Docker Image "kpjavaspringboot-**docker"** Successfully Created! & Running Inside Docker Container Successful..
- 

- Docker Containers : '**kpreactjswebapp-docker**' Apps Successfully Running inside a Docker Container! : http://localhost:8181
- http://localhost:8181/greet

•


docker desktop PERSONAL
Containers
View all your running containers and applications. Learn more
Container CPU usage 0.19% / 800% (8 CPUs available)
Container memory usage 155.4MB / 1.87GB
Name Image Container ID Port(s) CPU... Last started Actions
welcome-to-docker welcome-to-docker 791b5ca43b4b 8088:3000 0% 1 day ago
kpreactjswebapp-docker kpreactjswebapp-docker 8c3c005bd7c8 3001:3000 0% 1 day ago
kpjavaspringboot-docker kpjavaspringboot-docker 58f1f602583d 8181:8080 0.27% 33 seconds ago
Showing 3 items
Engine running
RAM 1.05 GB CPU 0.13% Disk: 7.51 GB used (limit 1006.85 GB)
KP Professional Workspace
GET http://localhost:8181/greet
200 OK 103 ms 252 B
KP : KPJavaSpringBootApplication Docker Container : KPCRUDController (REST Controller!)